

Analytics Dashboard (User & Quiz Analytics)

1. Feature Name

Analytics Dashboard (User & Quiz Analytics)

2. Purpose

- **Problem Solved:** Learners need insight into their own progress, streaks, and completion rates; instructors/admins need per-quiz performance data (pass rates, score distributions, hardest questions).
 - **Why It Exists:** The dashboard gives an at-a-glance summary (stats, per-path progress, recent activity), while the analytics layer provides deeper longitudinal metrics and quiz-level performance breakdowns.
 - **Role in Application:** It is the measurement/reporting subsystem. It aggregates data already collected by Progress, QuizAttempt/StudentAnswer, Certificate, and UserClassroom records.
-

3. What the User Can Do

Any authenticated user

- View a personal dashboard: enrolled-path count, completed-module count, active classrooms, and certificates earned.
- See per-path module progress (completed vs. total) and a recent-activity feed (last 10 module completions).
- View personal analytics: total modules completed, paths enrolled, certificates, active classrooms, current streak, average completion rate.
- See a 7-day weekly activity chart (modules completed per day) and a module-type breakdown (content type vs. completed count).
- See per-path completion percentages.

Admin only

- View analytics for any quiz: total attempts, unique students, average score, pass/fail counts, pass percentage.
- See a 5-bucket score distribution (0–20%, 21–40%, 41–60%, 61–80%, 81–100%).
- See per-question statistics (times answered, times correct, success rate, difficulty) plus the most-incorrect and hardest questions.

4. Feature Workflow

[Learner opens dashboard]

- GET /api/v1/dashboard (JWT → userId)
- DashboardService.GetDashboardDataAsync(userId)
- Enrolled paths ← Progresses (distinct Module.LearningPathId)
- Stats (completed modules, classrooms, certificates) ← counts
- Per-path progress + recent 10 completions
- Rendered as **stat** cards, progress bars, activity list

[Learner opens analytics]

- GET /api/v1/analytics (userId)
- AnalyticsService.GetUserAnalyticsAsync(userId)
- Streak computed from completed dates; 7-day weekly grid built
- Per-path completion %, module-type breakdown, average rate

[Admin opens quiz analytics]

- GET /api/v1/quizzes/{quizId}/analytics (Admin role required)
- AnalyticsService.GetQuizAnalyticsAsync(quizId)
- Aggregate attempts (Status >= Submitted) into score metrics, distribution buckets, and per-question success rates

5. How It Works Internally

Dashboard Data (DashboardService)

GetDashboardDataAsync(userId):

- `enrolledPathIds` — distinct `Progress.Module.LearningPathId` for the user (paths the user has progress in).
- `completedModules` — count of `Progresses` with `IsCompleted == true`.
- `activeClassrooms` — count of `UserClassrooms` rows.
- `certificates` — count of `Certificates` rows for the user.
- `pathProgress` — per enrolled path: `TotalModules = lp.Modules.Count`, `CompletedModules = count of modules having a completed Progress for the user`.
- `recentActivity` — last 10 completed modules (`CompletedAt != null`, ordered desc), each rendered as `"Completed module: {Module.Title}"` with a `MMM dd, yyyy` date string. `Type = "completion"`.

User Analytics (AnalyticsService.GetUserAnalyticsAsync)

- **Core counts:** completed modules, enrolled paths (distinct `LearningPathId`), certificates, active classrooms.
- **Weekly activity (last 7 days):** `sevenDaysAgo =.UtcNow.AddDays(-6).Date`; fetch completed `Progress.CompletedAt` dates $\geq$ that; build an array of 7 `WeeklyActivityDto` (day name `ddd`, count of completions that day) so empty days appear as zero.
- **Streak:** `CalculateStreak` walks backward day-by-day from `UtcNow.Date`, counting consecutive days present in the completed-date set; stops at the first missing day. A day counts if the user completed at least one module.
- **Path completions:** for each enrolled path, completed-module count vs. total; `Percent = round(completed/total*100)`; zero if total is 0.
- **Module type breakdown:** groups completed progresses by `Module.ContentType` and counts.
- **Average completion rate:** mean of the per-path `Percent` values (rounds to 1 decimal; 0 if no paths).

Quiz Analytics (`AnalyticsService.GetQuizAnalyticsAsync`)

- Loads the quiz (throws `KeyNotFoundException` if missing $\rightarrow$ controller returns 404).
- **Attempts:** all `QuizAttempts` with `Status >= AttemptStatus.Submitted`; `scoredAttempts` = those with `Percentage.HasValue`.
- **Metrics:** `totalAttempts = scoredAttempts.Count`; `uniqueStudents` = distinct `UserId` across submitted attempts; `avgScore = round(Average(Percentage), 1)`; passed/failed from `Passed` flag; `passPct = round(passed/total*100, 1)`.
- **Score distribution:** five buckets (`<=20`, `21-40`, `41-60`, `61-80`, `>80`) counted against the scored attempts.
- **Question analytics:** for every question in the quiz's `QuestionBankId`: `timesAnswered / timesCorrect` from `StudentAnswers` (joined to their `Option`, restricted to attempts of this quiz with `Status >= Submitted`); `successRate = round(correct/answered*100, 1)`; question text truncated to 100 chars + "...".
- **Most incorrect / hardest:** both derived as the question with the lowest success rate (`OrderBy(SuccessRate).First()`). Note: both fields currently point to the same question.

6. Frontend Implementation

Layer	File Path	Responsibility
Page	<code>frontend/src/pages/dashboard/DashboardPage.tsx</code>	Renders stats cards, path progress bars, recent activity feed
Page	<code>frontend/src/pages/analytics/AnalyticsPage.tsx</code>	Renders user analytics (weekly chart, streaks, breakdowns)

Layer	File Path	Responsibility
Service	frontend/src/services/analyticsService.ts	Calls dashboard + analytics endpoints
State	frontend/src/redux/slices/ (analytics/dashboard slices)	Client-side caching of analytics data

7. Backend Implementation

Layer	File Path	Responsibility
Controller	backend/Controllers/DashboardController.cs	GET /api/v1/dashboard (any authenticated user)
Controller	backend/Controllers/AnalyticsController.cs	GET /api/v1/analytics (user), GET /api/v1/quizzes/{quizId}/analytics (Admin)
Service	backend/Services/Dashboard/DashboardService.cs	Dashboard aggregation
Service	backend/Services/Analytics/AnalyticsService.cs	User + quiz analytics, streak, distributions
Interface	backend/Interfaces/Services/IDashboardService.cs	Dashboard contract
Interface	backend/Interfaces/Services/IAalyticsService.cs	Analytics contract
DTO	backend/DTOs/Dashboard/DashboardResponseDto.cs	Dashboard response models
DTO	backend/DTOs/Analytics/AnalyticsDto.cs	User/quiz analytics response models

8. Database Implementation

Read-only aggregation over existing tables — no analytics-specific tables or migrations:

Source Table	Used For
Progresses	Enrolled paths, completed modules, streak, weekly activity, per-path %, module-type breakdown, recent activity
UserClassrooms	Active classroom count
Certificates	Certificate count
Quizzes / QuestionBanks	Quiz identity + question scope for quiz analytics
QuizAttempts	Total attempts, unique students, avg score, pass/fail, distribution
StudentAnswers (+ Options)	Per-question success rates

9. Security & Authorization

- `DashboardController` and `AnalyticsController` are `[Authorize]`; user identity comes from the `JWT ClaimTypes.NameIdentifier`.
- `GET /api/v1/quizzes/{quizId}/analytics` additionally requires the **Admin** role (`[Authorize(Roles = "Admin")]`).
- Users can only see their own progress/analytics (all queries are filtered by `userId`).
- Missing quiz → `KeyNotFoundException` → 404 with an error payload (not an exception leak).

10. Important Business Rules

1. **Streak** counts only consecutive calendar days ending today or earlier — any gap resets it to 0.
2. **Weekly activity** always returns exactly 7 entries (last 7 days), zero-filled for inactive days.
3. **Score distribution buckets** are exclusive-of-upper-bound except the top bucket: `0-20`, `21-40`, `41-60`, `61-80`, `81+`.
4. **Pass percentage** is based on scored attempts only (`Percentage.HasValue`).
5. **Most incorrect / hardest question** = the question with the lowest success rate (same question in both fields).
6. **Question text** in analytics is truncated to 100 characters.
7. **Average completion rate** is the mean of per-path percentages (paths with 0 modules contribute 0%).

11. Example of Internal Execution

Step 1: Learner views dashboard

- User opens the Dashboard; `GET /api/v1/dashboard` resolves `userId` from the JWT.
- The service computes enrolled paths (2), completed modules (14), active classrooms (1), certificates (0), per-path progress (Path A 8/10, Path B 6/8), and the 10 most recent completions.
- Frontend renders 4 stat cards, two progress bars, and an activity list.

Step 2: Streak calculation

- The user completed modules on Aug 10, 11, 12, 13 (UTC). `CalculateStreak` walks Aug 13 → 12 → 11 → 10 (all present) then Aug 9 (missing) → streak = 4.

Step 3: Admin inspects a quiz

- `GET /api/v1/quizzes/5/analytics`: 120 submitted attempts → avg 64.2%, 81 passed (67.5%), 39 failed. Distribution: 0–20%:5, 21–40%:12, 41–60%:28, 61–80%:41, 81–100%:34.
- Question analytics show Q#7 at 32% success → surfaced as both most-incorrect and hardest.

12. Files Involved

Frontend

- `frontend/src/pages/dashboard/DashboardPage.tsx`
- `frontend/src/pages/analytics/AnalyticsPage.tsx`
- `frontend/src/services/analyticsService.ts`
- Dashboard/analytics Redux slices

Backend

- `backend/Controllers/DashboardController.cs`
- `backend/Controllers/AnalyticsController.cs`
- `backend/Services/Dashboard/DashboardService.cs`
- `backend/Services/Analytics/AnalyticsService.cs`
- `backend/Interfaces/Services/IDashboardService.cs`
- `backend/Interfaces/Services/IAalyticsService.cs`
- `backend/DTOs/Dashboard/DashboardResponseDto.cs`
- `backend/DTOs/Analytics/AnalyticsDto.cs`

Database

- No dedicated analytics tables — queries `Progresses`, `UserClassrooms`, `Certificates`, `Quizzes`, `QuizAttempts`, `StudentAnswers`, `Options`.
-

13. Functionality

- Personal dashboard with four summary stats
- Per-learning-path module progress
- Recent activity feed (last 10 completions)
- 7-day weekly activity chart (zero-filled)
- Daily streak calculation (consecutive completion days)
- Average completion rate across paths
- Per-path completion percentages
- Module-type breakdown of completed content
- Per-quiz attempt/pass/score aggregation (Admin)
- Five-bucket score distribution (Admin)
- Per-question success-rate analytics (Admin)
- Most-incorrect / hardest question identification (Admin)